Nidaan — IA-II Activity 1 submission checklist

Deadline: **Sunday 11 October 2026, 11:59 PM IST.** Recorded runs must be presented as recorded (date + run ID), never as live. No step below requires paid API use.

1. Assignment sources and how conflicts were handled

Sources: `docs/assignment/AI – IA2 Instructions.pdf` (activity sheet), `docs/assignment/AI IA2 report format.pdf` (report template), `docs/assignment/IA2 Instructions.pdf` (lecture slides), `docs/assignment/IA2_Activity1_Announcement.md` (professor's announcement, verbatim).

Point	What the sources say	How the report handles it
Group size	Sheet and announcement: 3 students, same lab batch. Template cover shows 4 "Roll No: Name" lines	3 lines on the cover (group of 3)
Part C	Sheet and announcement require Part C – Analysis & Comparison (rule-based vs LLM-driven, static vs dynamic planning, no memory vs contextual memory, single- vs multi-step). The template has no Part C heading, only "Discussion, takeaway"	Explicit **Part C** section placed before "Discussion, takeaway"
Number of tools	Sheet/announcement: "at least two tools"; slides: "Tools integrated with agents (choose at least ONE)"	Ten tools — satisfies both
Framework	Sheet: "at least ONE" of the listed (or equivalent); slides: "choose ONE"	LangChain + LangGraph (ReAct-style); LangChain is on both lists
Tool types	Lists web search, calculator, calendar, documents, CSV/JSON, Python tools; "use something equivalent"	Custom Python tools over a simulated environment ("Python tools" category); stated in the report
Workflow to demonstrate	Sheet: "User Goal → Agent Reasoning → Tool Call → Observation → Next Action → Final Answer"; slides: "Goal → Plan → Tool Use → Final Output"	Tool-flow diagram + log excerpt of the hero run; hidden reasoning is not stored, the observable reasoning is plans + `why`
Template wording	Template cover: "Department of Electronics & Computer Engineering", "Academic Year 2026-27"; "Demo will be taken in lab duration"	Kept verbatim; checked against `AI IA2 report format.pdf` on 2026-10-11 (department line and academic year match the template exactly)
Rubric	Modern tools & frameworks 5; reasoning & tool orchestration 5; architecture & design clarity 5; analysis & reflection 5	Covered by B.1–B.2, B.3–B.5, Part A + diagrams, Part C + Discussion

2. Requirement → evidence

Requirement	Report section	Evidence (file)	Status
Goal / problem statement	Introduction; Part A "Goal"	`nidaan/prompt.py` (GOAL)	Done
Agent role	Part A "Agent role"	`nidaan/graph.py`	Done
Tools used (≥ 2) + justification	Tools line; Part A; B.2	`nidaan/tools.py`, `settlesim/tools.py`	Done (10 tools)
Memory type (short-term/context)	Part A "Memory type"	`nidaan/graph.py` (`add_messages`), `nidaan/plan.py`	Done (no persistent memory claimed)
Planning strategy (ReAct / step-by-step)	Part A "Planning strategy"	`nidaan/plan.py`, `nidaan/tools.py` (`update_plan`)	Done
LLM + justification	B.1	`config.py`, `nidaan/llm.py`	Done
One LLM-based agent, multi-step reasoning	B.3–B.5	hero log `runs/hero-s323-20261010T122852.jsonl`	Done
Architecture diagram	B.3 (Mermaid, rendered in PDF)	`docs/Nidaan_IA2_Report.md`	Done
Tool-flow diagram	B.4 (Mermaid, rendered in PDF)	same	Done
Observable behaviour (logs/traces)	B.5 log excerpt	`runs/*.jsonl`	Done (verbatim excerpt)
Prompts	B.5 (system prompt verbatim; goal in Part A)	`nidaan/prompt.py`, `baselines/oneshot.py`	Done

Output screenshots	B.5 Figures S1, S2, S3	`docs/screenshots/`	Done — embedded in the PDF (S1, S2 on page 6; S3 on page 7). S1 terminal timeline of `hero-s323-20261010T122852`; S2 current Recovery Verification of `nidaan-hero-s323-r1-20261010T171521075447` (VERIFY #37, verbatim transcription under it); S3 current Live Execution after live Runbook run `live-runbook-hero-s323-20261011T121624409367`. See §4.
Code repository	B.5 "Code repository / notebook"	repository, `README.md`	Done — link in B.5: https://github.com/akshay-ashwin/Nidaan-Agentic-Incident-Response — **Manual input:** push so the link resolves to the final code
Demo of working agent	§3 below	CLI + dashboard	Script ready — **Manual input:** rehearse for the lab
Classical vs modern comparison (4 criteria)	C.1 table	`baselines/runbook.py`, results	Done
Actual results incl. failures and limits	C.2–C.4, Discussion	`results/*.json`, `reports/*.md`	Done
Conclusion, references	Conclusion, References	—	Done
PDF 5–10 pages	—	`docs/Nidaan_IA2_Report.pdf`	Done: final rebuild 2026-10-11, **10 pages** (cover included; at the limit — page 10 holds only References), three screenshots, no placeholders

Status summary

Completed (verified locally): report content for every required section; architecture and tool-flow diagrams; verbatim prompt and log excerpt; code, 360 offline tests passing (no LLM calls; they do not establish live success); runbook 6/6 and one-shot 1/6 on the six dev scenarios; Nidaan hero VERIFIED with the original prompt and with revision 2; Nidaan dev_f2 FAILED (original) and VERIFIED (revision 1); five-section dashboard with explicit live Runbook/Nidaan runs (live runs in `runs_live/`, excluded from all metrics); screenshots S1 (terminal), S2 and S3 (current dashboard) embedded; PDF rebuilt on 2026-10-11 at 10 pages (diagrams rendered, no placeholders); `README.md`.

Incomplete but disclosed in the report:

- the report-field descriptions added after the revision-2 hero run have not been used in any evaluation run (one live dashboard demonstration on 2026-10-11 used them and was VERIFIED; it is

reported separately and is not evaluation evidence);
- Nidaan has not been evaluated on all six development scenarios with the latest code (never run on dev_f4, dev_f7, dev_f1_f2, dev_f4_f6 — missing, not failed; dev_f2 not run with revision 2);
- the ten-run hero rehearsal (≥ 9 successes) has not been performed;
- the held-out evaluation has not been performed (held-out material sealed);
- replanning remains a measured limitation (3 of 13 failures followed by a plan update within two steps).

Done on 2026-10-11: names and roll numbers filled (Akshay Ashwin 16015024005, Saanvi Baraskar 16015024007, Ameya Deore 16015024016); department line verified against the template; report aligned to the template (red "Group of 3…"/"Demo…" lines, B.5 sub-items in template order, centered Discussion/Conclusion/References headings, cover side bar); repository link inserted; dashboard, test count (360) and Figures S2/S3 updated to the current UI; PDF rebuilt and checked (10 pages).

Requires manual input from you: Git commit/push (including the gitignored run logs if they must be in the repository); submission; demo rehearsal.

Known differences from the template (deliberate): 3 name lines instead of 4 (group of 3); an extra *Part C – Analysis & Comparison* section before "Discussion, takeaway" (required by the activity sheet); Discussion, Conclusion and References are not forced onto separate pages (doing so produced 11 pages, over the 10-page limit); the template's decorative back page is not reproduced; the footer appears on every page.

3. Demo script (≈10 minutes, lab)

All commands from the repository root.

1. **Tests (offline, ~5 s):** `.venv/bin/python -m pytest -q` → expected `360 passed`.
2. **Live environment, no LLM (~1 s):**

   ```
   .venv/bin/python scripts/run_dev_eval.py --arm runbook --repeats 1 --scenarios
   hero --runs-dir /tmp/nidaan_demo/runs --out
   /tmp/nidaan_demo/runbook_hero_$(date +%H%M%S).json
   .venv/bin/python scripts/analyse_traces.py --log /tmp/nidaan_demo/runs/*.jsonl
   --print-timeline
   ```

   Expected: `goal_success=True duplicates=0 final=GOAL_MET`.
3. **The agent's verified hero run (recorded — say so):** `.venv/bin/python scripts/analyse_traces.py --log runs/hero-s323-20261010T122852.jsonl --print-timeline`
4. **Dashboard:** `.venv/bin/streamlit run ui/app.py`. Loading it starts nothing. Show *Overview*, then *Recovery Verification* → *Recorded run* (sidebar run `hero · nidaan · VERIFIED · nidaan-hero-s323-r1-…`), then *Trace & Evaluation*. Optionally click **Run Runbook** in *Investigate & Run* (no model calls; logged to `runs_live/`). **Run Nidaan** uses API quota and needs the confirmation box; live runs are never evaluation evidence.
5. **Failure case:** in the sidebar *Recorded run*, select `dev_f2 · nidaan · FAILED · nidaan-dev_f2-s11-r1-20261010T130437328361`; in *Trace & Evaluation* → *Run trace*, show #39 and the step-limit FINAL.
6. **Optional live agent run** — only if the Groq console shows ≥ ~55,000 tokens left today: `.venv/bin/python scripts/run_dev_eval.py --arm nidaan --repeats 1 --scenarios hero --out /tmp/nidaan_demo/nidaan_hero_demo.json --runs-dir /tmp/nidaan_demo/runs` (writes outside `results/` and `runs/`, so the recorded result `results/dev_nidaan_hero_r1_prompt-812cea17.json` is never overwritten) (≈4–10 min; uses

prompt revision 2; the outcome is not guaranteed; never substitute the recorded run if it fails). The same can be started from the dashboard with **Run Nidaan** (logged to `runs_live/`, not evaluation evidence).

4. Screenshots in the report (real runs only)

All three are captured and embedded (S1, S2 on page 6; S3 on page 7). To recapture one, use the steps below and keep the file name; the PDF build embeds `docs/screenshots/<file>` automatically.

Figure	File	What it shows	How to capture
S1	`S1_hero_timeline.png`	Terminal: trace timeline of the recorded hero run `hero-s323-20261010T122852` (original prompt; VERIFIED at #41)	Run demo step 3; screenshot the terminal with the command line visible
S2	`S2_dashboard_verification.png`	Current dashboard, *Recovery Verification*: recorded revision-2 hero run `nidaan-hero-s323-r1-20261010T171521075447` (VERIFY #37): agent report, independent verifier (V1–V3 passed) and simulator goal check side by side	`streamlit run ui/app.py` → sidebar *Recorded run* = `hero · nidaan · VERIFIED · nidaan-hero-s323-r1-…` → *Recovery Verification* → *Recorded run*; capture the verdict banner and the three panels
S3	`S3_dashboard_live_runbook.png`	Current dashboard, *Live Execution* after a live Runbook run of hero (`live-runbook-hero-s323-20261011T121624409367`): goal met, 20/20 payouts sent once, 0 duplicates; labelled as not part of the evaluation	*Investigate & Run* → scenario `hero` → **Run Runbook** (no model calls); the page switches to *Live Execution*; capture the live-run label, banner, stats and payout ledger

`S2_dashboard_trace.png` in the same folder is the pre-redesign dashboard screenshot; it is no longer used in the report. Optional extra evidence (not embedded): `pytest` output (demo step 1); the dev_f2 failure trace (demo step 5).

5. Manual actions remaining (in order)

1. Names, roll numbers, department and repository link are already in the report; nothing to fill in.
2. Only if the report source changes: rebuild the PDF. The project `.venv` does not contain the builder's packages, so use a throwaway environment:

```
python3 -m venv /tmp/nidaan_pdfenv && /tmp/nidaan_pdfenv/bin/pip install
markdown==3.7 pypdf==5.1.0
/tmp/nidaan_pdfenv/bin/python docs/build_report_pdf.py
```

It prints the page count (must stay 5–10; currently 10, so any addition needs a matching cut). Needs Google Chrome and a network connection (mermaid.js from a CDN).

3. Open `docs/Nidaan_IA2_Report.pdf` and check the cover, diagrams (page 4), screenshots (pages 6–7) and tables.
4. Commit and push (you handle Git). `runs/` is gitignored: force-add the evidence logs if the repository must contain them (`git add -f runs/hero-s323-20261010T122852.jsonl runs/nidaan-hero-s323-r1-20261010T171521075447.jsonl runs/nidaan-dev_f2-* runs/runbook-* runs/oneshot-*`), or attach them. `runs_live/` is also gitignored; the report cites two live logs (`runs_live/live-nidaan-hero-s323-20261011T103723679527.jsonl` , `runs_live/live-runbook-hero-s323-20261011T121624409367.jsonl`), so force-add those too if the repository must contain them. Also add the new untracked files (`README.md` , `docs/` including `docs/screenshots/S2_dashboard_verification.png` and `S3_dashboard_live_runbook.png` , `results/` , `reports/` , new modules/tests).
5. Submit the PDF before 11:59 PM IST; rehearse the demo (§3) for the lab session.